

****Pornind de la aplicația exemplu din laboratorul 10, să se modifice aceasta astfel încât să se introducă un procesor de flux care să permită împreună cu alte microservicii (care vor trebui eventual create) să se inițieze o discuție privată între câțiva utilizatori (deci ne mai trebuie cel puțin un procesor master și unul de comunicație replicat pentru fiecare utilizator). Se vor utiliza microservicii Python și o comunicare prin RabbitMQ. Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod.

NUSH CE EXEMPLU

1. RABBIT MQ - lab 5 pt config: pip install pika, fisier in care se face comunicarea cu Rabbit MQ este in lab 5 la ex 1 la python tinkter
2. Install in python plugins or smt ce are treaba cu kafka
File → Settings → Project: NUME_PROIECT → Python Interpreter
→ apăsați pe semnul plus din dreapta (Install) și căutați „kafka-python”.
Apăsați pe „Install Package” după ce ați selectat modulul din lista de rezultate.
pt exemplu cu python kafka lab 10 (nu cred ca de asta vb sincer)

GASIT

ADAUGAT FISIER RabbitMQConect.py

POTI SA TE INSPIRI DIN LAB5 exemplu 2 din python

```
import pika
from retry import retry
```

Interfata pentru Producer si pentru Consumer

class RabbitMqInterface:

```
    def __init__(self):
        self.config = {
            'host': '0.0.0.0',
            'port': 5678,
            'username': 'student',
            'password': 'student'
        }
        self.list_msg = []
        self.credentials = pika.PlainCredentials(self.config['username'], self.config['password'])
        self.parameters = (pika.ConnectionParameters(host=self.config['host']),
```

```
pika.ConnectionParameters(port=self.config['port']),
pika.ConnectionParameters(credentials=self.credentials))
```

Routing key ofera date despre coada spre care se trimite

```
class RabbitMqProducer(RabbitMqInterface):
```

```
    def __init__(self, exchange: str, routing_key: str):
        super().__init__()
        self.config["exchange"] = exchange
        self.config["routing_key"] = routing_key
```

```
    def send_message(self, message):
```

```
        # automatically close the connection
        with pika.BlockingConnection(self.parameters) as connection:
            # automatically close the channel
            with connection.channel() as channel:
                channel.basic_publish(exchange=self.config['exchange'],
                                     routing_key=self.config['routing_key'],
                                     body=message)
```

Rabbit_queue este coada din care se citesc informatii

```
class RabbitMqConsumer(RabbitMqInterface):
```

```
    def __init__(self, rabbit_queue: str):
        super().__init__()
        self.config["queue"] = rabbit_queue
        self.connection = pika.BlockingConnection(self.parameters)
        self.channel = self.connection.channel()
        self.channel.queue_purge(self.config["queue"])
```

```
@retry(Exception, delay=1, tries=15)
```

```
def receive_message(self):
```

```
    try:
        result_msg = self.channel.basic_get(self.config['queue'])
        if result_msg[2]:
            self.list_msg.append(result_msg[2].decode("utf-8"))
            self.channel.basic_ack(result_msg[0].delivery_tag)
        else:
            raise Exception("nici o valoare noua in coada.")
    # Don't recover connections closed by server
    except pika.exceptions.ConnectionClosedByBroker:
        print("Connection closed by broker.")
    # Don't recover on channel errors
    except pika.exceptions.ConnectionClosedByBroker:
```

```
        print("AMQP Channel Error")
except ValueError:
    print("Value error!")
# Don't recover from KeyboardInterrupt
except KeyboardInterrupt:
    print("Application closed.")
```